



TRABALLO FIN DE GRAO
GRAO EN CIENCIA E ENXEÑARÍA DE DATOS

Visualizando el aprendizaje máquina: de la abstracción al entendimiento

Estudiante Héctor Aldao Amoedo
Dirección: Alejeandro Rodriguez Árias

A Coruña, junio de 2026.

Dedicatoria¿?

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. ¿?

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Palabras clave:

- First itemtext
- Second itemtext
- Last itemtext
- First itemtext
- Second itemtext
- Last itemtext
- First itemtext

Keywords:

- First itemtext
- Second itemtext
- Last itemtext
- First itemtext
- Second itemtext
- Last itemtext
- First itemtext

Índice general

1	Introducción	1
1.1	Contexto y motivación	1
1.1.1	Definición del Problema	1
1.1.2	Objetivos	2
1.1.3	Alcance y Limitaciones	2
1.1.4	Metodología de Trabajo	2
1.1.5	Estructura de la Memoria	2
2	Estado del Arte	3
2.1	Recursos audiovisuales explicativos	3
2.2	Simuladores interactivos de redes neuronales	4
2.3	Simuladores interactivos de árboles de decisión	5
2.4	Oportunidades de mejora	5
3	Herramientas y técnicas	6
3.1	Godot como motor de desarrollo	6
3.2	Lenguajes de programación empleados	8
3.2.1	GScript como base del proyecto	8
3.2.2	Python para validación de resultados	8
3.2.3	Rust para generación de fórmulas	9
3.2.4	JavaScript para integración con el navegador	9
3.3	Control de versiones y despliegue	9
4	Metodología y planificación	10
5	Desarrollo	11
5.1	Mockup: versión 0.1	11
5.1.1	Primeras escenas	11

5.1.2	Exportación web con GitHub Pages	12
5.2	Árbol de decisión: versión 0.2	12
5.2.1	Primera arquitectura del árbol	12
5.2.2	Reorganización de la vista y primeros elementos explicativos	17
6	Conclusiones	18
7	Trabajo futuro	19
A	Material adicional	21
	Bibliografía	24

Índice de figuras

5.1	Versión preliminar del menú principal.	13
5.2	Primera escena de prueba para ver el funcionamiento del árbol.	13

Índice de cuadros

Capítulo 1

Introducción

PRIMER capítulo da memoria, onde xeralmente se exporán as liñas mestras do traballo, os obxectivos, etc. Incluimos un par de exemplos de citas e de referencias

LA inteligencia artificial ...?

1.1 Contexto y motivación

Los algoritmos de aprendizaje máquina son técnicas matemáticas que escapan de las fronteras de lo que es entendible a simple vista por ¿una persona cualquiera.¿ Desde sus raíces más puramente matemáticas hasta su implementación tanto hardware como software.

Esto ya era cierto en los tiempos de los primeros algoritmos de aprendizaje ¿por refuerzo, árboles o qué?, pero con el surgir de los modelos de aprendizaje profundo, esta realidad es más palpable que nunca.

Bajo estas premisas es que se siente indispensable ¿proporcionar a la gente los materiales y métodos para que puedan comprender lo mejor posible esta tecnología cada vez más omnipresente¿. ¿Y a de más el aprendizaje es mejor si es interactivo ¿?

1.1.1 Definición del Problema

Hoy en día las principales formas de aprender cómo funcionan los modelos de inteligencia artificial (sobre todo los clásicos, pero también los modernos) es a través de los siempre confiables libros y universidad, las más ancestrales fuentes de conocimiento. Entre los formatos más actuales también encontramos los vídeos divulgativos en plataformas online como Youtube, donde la posibilidad de que todo el mundo comparta su contenido ha dado lugar a ¿muchas oportunidades de ver las interpretaciones visuales de otras personas¿.

Sin embargo ¿aún no existe ninguna herramienta pedagógica que permita interactuar en tiempo real con los algoritmos.¿ ¿Programándolos se pueden ver sus resultados, su evolución en una gráfica, pero no palpar los que está sucediendo en las entrañas de la bestia mientras

mastica el equivalente a su comida: los datos, tanto en entrenamiento como en inferencia;?
((.....))

1.1.2 Objetivos

((Crear la aplicación - paso 1, hacerla - paso 2, exportarla
Publicarla para que todo el mundo la pueda disfrutar))
((creo qe hay q se rmás concreto))

Los objetivos que se plantean en este trabajo comienzan por plantearse qué algoritmos de IA son los más interesantes para ¿explicar¿. Esta es, en última instancia, una cuestión subjetiva, por lo que en este trabajo se abordó esta decisión mirando ¿de cara¿ a dos de los mayores paradigmas de la IA: la simbólica y el deeplearning. ((no se si decir de forma pragmática y decir que tenía en mente las redes neuronales, y que los árboles se parecían lo suficiente, o poner un poco de fantasía de "los árboles, uno los primeros amigos del hombre en cuánto a lo que hacer que una máquina aprenda a jugar al ajedrez..." (lo

De ambos se decidió que los mejores candidatos para protagonizar una explicación que busque asentar los fundamentos eran las versiones más fundamentales de cada uno. Del lado de la IA simbólica está el árbol de decisión ((planteado por bla bla bla en blabal)), conocido por ser ... white box Y en el deeplearning, el ladrillo fundacional, la red neuronal ((lo mismo de histoia...))

1.1.3 Alcance y Limitaciones

1.1.4 Metodología de Trabajo

1.1.5 Estructura de la Memoria

Estado del Arte

EL aprendizaje profundo se apoya en conceptos matemáticos y computacionales que no siempre resultan fáciles de visualizar: funciones de activación, pesos, sesgos, propagación hacia delante, retropropagación o gradientes, entre otros. Esta dificultad ha motivado el desarrollo de recursos educativos orientados a hacer más comprensible el comportamiento de las redes neuronales mediante explicaciones visuales y simulaciones interactivas. En el contexto de este trabajo, resultan especialmente relevantes las herramientas que permiten comprender el proceso de aprendizaje de los modelos y la relación entre sus componentes internos.

2.1 Recursos audiovisuales explicativos

Uno de los recursos divulgativos más conocidos para introducir las redes neuronales es la serie de Grant Sanderson “Neural networks” [1], publicada en el canal de YouTube 3Blue1Brown. Este proyecto cuenta también con una página web [2], cuya principal aportación pedagógica es la posibilidad de interactuar con una red neuronal entrenada con el conjunto de datos MNIST [3]. La herramienta permite observar en tiempo real cómo la red reacciona a los cambios en el dato de entrada, representado mediante una superficie de dibujo en la que cada píxel se corresponde con una neurona de la capa de entrada.

Este tipo de recurso resulta valioso para desarrollar intuición, ya que reduce la barrera de entrada matemática y proporciona una primera aproximación visual al funcionamiento de una red.

Su formato es fundamentalmente expositivo. En la página web, dos de las diez lecciones incorporan un componente interactivo. Dicha interacción se orienta siempre a la relación entre la entrada y la salida de una red ya entrenada. El estudiante observa una explicación apoyada en recursos visuales, limitada al conjunto MNIST y sin capacidad de experimentar de forma directa con la arquitectura del modelo. Por ello, aunque es adecuado para motivar y

contextualizar el problema, no cubre por completo la necesidad de aprendizaje activo asociada a la comprensión del efecto de cada decisión de diseño en el comportamiento de una red neuronal.

2.2 Simuladores interactivos de redes neuronales

Frente al formato audiovisual, existen herramientas que permiten manipular redes neuronales en tiempo real. Un ejemplo es “A Neural Network Playground”, desarrollado por Daniel Smilkov y Shan Carter y publicado por TensorFlow [4]. Esta aplicación permite configurar una red neuronal sencilla desde el navegador, seleccionar conjuntos de datos sintéticos, modificar hiperparámetros y observar cómo evoluciona la frontera de decisión durante el entrenamiento.

La relevancia de TensorFlow Playground reside en que conecta elementos habitualmente tratados de forma abstracta con consecuencias observables. Cambiar el número de capas, el número de neuronas, la tasa de aprendizaje, la regularización o las características de entrada altera de manera visible el proceso de ajuste. Esto facilita que el estudiante relacione conceptos como sobreajuste, generalización o convergencia con resultados palpables.

La herramienta se centra principalmente en la frontera de decisión y en la pérdida, sin profundizar de forma detallada en operaciones internas como el cálculo de gradientes o la propagación de errores capa a capa.

Otra línea relevante está formada por las aplicaciones centradas en arquitecturas específicas. “GAN Lab”, desarrollado por Minsuk Kahng, Nikhil Thorat, Polo Chau, Fernanda Viégas y Martin Wattenberg, permite experimentar en el navegador con redes generativas adversarias [5]. Este tipo de modelo introduce una dinámica de aprendizaje distinta a la de una red supervisada clásica, ya que enfrenta dos redes: un generador, que intenta producir muestras plausibles, y un discriminador, que intenta distinguir entre datos reales y generados.

La aportación principal de GAN Lab es representar visualmente una interacción compleja como la de un par de redes adversarias. La herramienta permite observar cómo cambian las distribuciones generadas durante el entrenamiento y cómo la competencia entre generador y discriminador afecta al resultado. Desde el punto de vista educativo, esto resulta útil para explicar que no todos los modelos de aprendizaje profundo optimizan el mismo tipo de objetivo ni aprenden bajo el mismo esquema.

Su limitación principal es que presupone una base de conocimiento mayor que la requerida para una introducción a las redes neuronales. Para comprender correctamente una GAN es necesario haber asimilado previamente ideas como función de pérdida, optimización, representación interna o entrenamiento iterativo. En consecuencia, GAN Lab es una herramienta valiosa para una fase posterior del aprendizaje, aunque no sustituye a una aplicación centrada

en los fundamentos de una red neuronal básica.

2.3 Simuladores interactivos de árboles de decisión

En el ámbito de los algoritmos clásicos de aprendizaje automático, existen propuestas para ilustrar el funcionamiento de los árboles de decisión. La herramienta web interactiva sobre árboles de decisión de Interactive Machine Learning [6] permite al usuario construir un árbol y observar el proceso de división de un conjunto de datos en tiempo real.

La herramienta presenta una experiencia de uso limitada y ofrece explicaciones poco desarrolladas. Aunque muestra los valores de distintas métricas, no acompaña dichos valores con una interpretación suficiente de su significado ni de su papel dentro del algoritmo. Esto reduce su utilidad como recurso formativo autónomo, especialmente para estudiantes que se aproximan por primera vez a los árboles de decisión.

En comparación con las redes neuronales, la disponibilidad de herramientas interactivas centradas en árboles de decisión parece más reducida, especialmente cuando se buscan recursos que combinen visualización, explicación del criterio de partición y evaluación paso a paso.

2.4 Oportunidades de mejora

Desde la perspectiva de este proyecto, la principal oportunidad se encuentra en combinar las fortalezas de estos recursos. Una herramienta educativa eficaz debería mantener la claridad visual de los recursos divulgativos y su atención a la base matemática, incorporar la manipulación directa de los simuladores y ofrecer suficiente rigor técnico para que lo presentado al usuario se corresponda con el funcionamiento real del algoritmo.

Asimismo, debería evitar que la interacción se limite a modificar hiperparámetros globales. Para entender una red neuronal resulta especialmente útil poder observar el flujo de datos, la contribución de los pesos y el efecto del entrenamiento sobre cada componente del modelo.

Herramientas y técnicas

EL desarrollo de este trabajo se apoya en un conjunto de herramientas orientadas a construir una aplicación educativa, interactiva y ejecutable desde el navegador. La elección tecnológica tuvo en cuenta tanto criterios de implementación como la naturaleza del problema abordado: explicar modelos de aprendizaje automático mediante visualizaciones dinámicas, animaciones y ejemplos manipulables por el usuario. En consecuencia, se priorizaron tecnologías que facilitasen la construcción de interfaces interactivas, la validación de los algoritmos implementados y el despliegue web del resultado final.

3.1 Godot como motor de desarrollo

La herramienta principal empleada fue Godot, un motor de desarrollo libre y multiplataforma orientado a la creación de aplicaciones interactivas y videojuegos [7]. Aunque su uso más habitual se asocia al desarrollo de videojuegos, sus características también resultan adecuadas para una aplicación educativa como la desarrollada en este trabajo. El proyecto requiere presentar información visual, responder a acciones del usuario, animar procesos paso a paso y mantener una estructura clara entre los distintos elementos de la interfaz. Estas necesidades encajan de forma natural con el modelo de escenas, nodos y señales que propone Godot [8].

A lo largo de esta memoria se hablará en repetidas ocasiones de dos de las principales estructuras con las que se trabaja en Godot: las escenas y los nodos.

Los nodos son el elemento básico y más importante con el que se trabaja en el motor. Todo elemento y subelemento que vaya a estar presente en tiempo de ejecución en el programa es un nodo. Y cada nodo cuenta, principalmente, con una serie de atributos, métodos y señales. Igual que en la programación orientada a objetos, los atributos son variables o constantes y los métodos funciones. Las señales son disparadores de funciones. Cuando son emitidas llaman a la función a la que están conectadas. Son una forma de conectar eventos entre diferentes

nodos.

Los atributos, métodos y señales de cada nodo vienen definidos por su clase. Godot dispone de un gran árbol de clases predefinidas que heredan atributos y métodos de sus clases padre. Las tres clases básicas son: el “Node”, una clase vacía, la raíz del resto de nodos, que solo cuenta con los métodos mínimos para que funcione en el motor; el “Node2D”, característico por contar con atributos referentes a la posición en un espacio bidimensional; el “Node3D”, similar al anterior pero en un espacio tridimensional; y el “Control”, pensado para los elementos de interfaz de usuario. Por encima de estas, el desarrollador siempre puede definir sus propias clases ampliando las ya existentes.

Los nodos por sí solos son clases abstractas que no existen en el programa ejecutable final por sí mismos. Para instanciarlos están las escenas. Estas son la materialización concreta del proyecto: los archivos que son guardados en disco y con los que se interactúa en el editor. Están formadas, como mínimo, por un nodo raíz. De este nodo raíz pueden colgar nodos hijo, que a su vez pueden contar con sus propios nodos hijo, y así sucesivamente.

Finalmente, las escenas, como en última instancia son árboles de nodos, pueden ser instanciadas dentro de otras escenas. Esto habilita una modularidad total, puesto que toda estructura de nodos que sea susceptible de ser reutilizada, puede guardarse como una escena para no tener que ser recreada.

A mayores, los nodos pueden tener asociados un script. Estos son código desde el que se puede interactuar, principalmente, con los atributos y métodos del nodo al que están asociados, así como con sus hijos. Pero no están limitados a estos elementos. Siempre se puede acceder al nodo raíz, y desde este modificar cualquier propiedad global o acceder a cualquier nodo en la escena. O mediante la creación de “Autoloads” en el proyecto se puede acceder a clases personalizadas en cualquier punto del código.

La organización en escenas permitió separar componentes de la aplicación con responsabilidades distintas, como las vistas de entrenamiento, los paneles explicativos, las representaciones gráficas y los controles de interacción. Este enfoque favorece una estructura modular en la que cada parte de la interfaz puede desarrollarse, probarse y reutilizarse con menor acoplamiento. El sistema de señales facilita la comunicación entre elementos de la aplicación sin imponer dependencias directas innecesarias, lo que resulta útil en un entorno en el que las acciones del usuario deben provocar cambios visuales y actualizaciones del estado interno de los modelos.

La manera en que Godot organiza las interfaces mediante nodos resulta compatible con las representaciones habituales de los algoritmos tratados. Un árbol de decisión puede mostrarse como una estructura jerárquica de nodos conectados, mientras que una red neuronal puede representarse mediante capas, neuronas y conexiones. Esta correspondencia conceptual ayuda a trasladar las estructuras internas del algoritmo a elementos visuales manipulables dentro de

la aplicación.

Otro motivo importante para escoger Godot fue su capacidad de exportación a web. El motor permite generar una versión HTML5 de la aplicación [9], lo que evita exigir al usuario una instalación local y facilita el acceso desde un navegador. Esta característica es especialmente relevante en un proyecto con finalidad educativa, ya que reduce la fricción de uso y permite distribuir la herramienta como una aplicación web estática.

3.2 Lenguajes de programación empleados

El desarrollo combina varios lenguajes, cada uno empleado en una parte concreta del proyecto. La lógica principal de la aplicación se implementó dentro del propio entorno de Godot, utilizando GDScript para construir las visualizaciones, gestionar la interacción y ejecutar las implementaciones didácticas de los algoritmos. Junto a ello, Python, Rust y JavaScript se emplearon como tecnologías de apoyo para resolver necesidades específicas que no formaban parte del núcleo de la aplicación.

3.2.1 GDScript como base del proyecto

GDScript es el lenguaje oficial utilizado en Godot para definir la lógica de las escenas y la interacción entre nodos. Su integración directa con el motor permitió implementar tanto el comportamiento de la interfaz como las versiones didácticas de los algoritmos de aprendizaje automático. La implementación de estos algoritmos en el propio lenguaje de la aplicación facilitó la obtención de resultados parciales en cada paso del proceso, lo que permitió generar explicaciones detalladas y sincronizadas con la visualización.

3.2.2 Python para validación de resultados

Python se utilizó para contrastar el comportamiento de las implementaciones realizadas en la aplicación [10]. La finalidad de este uso fue disponer de un entorno externo con bibliotecas consolidadas que sirviesen como referencia para la lógica implementada en Godot. En particular, se compararon resultados producidos por la aplicación con los obtenidos mediante implementaciones estándar de árboles de decisión y redes neuronales en bibliotecas como scikit-learn [11] y PyTorch [12], respectivamente.

Este proceso ayudó a detectar errores tanto en la implementación de los algoritmos como en la conexión entre sus resultados internos y la información presentada por la interfaz. El uso concreto de Python en este proceso se detalla en el Capítulo 5.

3.2.3 Rust para generación de fórmulas

Rust se empleó para desarrollar un plugin para Godot destinado a generar fórmulas matemáticas de forma dinámica en tiempo de ejecución [13]. Esta necesidad surge porque la aplicación debe mostrar resultados numéricos y explicar los cálculos que conducen a ellos. Para ello se utilizó la biblioteca Ratex [14], que permite generar representaciones en LaTeX a partir de expresiones construidas durante la ejecución.

La elección de Rust se justifica por su rendimiento, su seguridad de memoria y su integración con Godot. En este proyecto, el complemento se compila a WebAssembly para poder utilizarlo en la versión web de la aplicación [15]. De este modo, la generación de fórmulas puede integrarse en el despliegue web sin depender de componentes nativos específicos de una plataforma. El desarrollo de este plugin y su integración con la aplicación se explican en el Capítulo 5.

3.2.4 JavaScript para integración con el navegador

JavaScript se utilizó en una parte concreta de la aplicación relacionada con la interacción con el navegador [16]. Su función principal fue permitir que el usuario cargase sus propios ficheros CSV, de forma que la herramienta no quedase limitada únicamente a conjuntos de datos predefinidos. Esta funcionalidad resulta importante desde el punto de vista educativo, porque permite experimentar con datos distintos y observar cómo cambian las decisiones del árbol o el comportamiento de la red en función de la entrada utilizada. De forma nativa, Godot permite cargar datos desde el sistema en el que se ejecuta la aplicación. En la exportación a HTML5, el funcionamiento de los navegadores modernos restringe esta característica, lo que hizo necesaria una capa de integración específica con JavaScript. La parte concreta desarrollada en JavaScript se describe en el Capítulo 5.

3.3 Control de versiones y despliegue

Para la gestión del código fuente se utilizó Git, un sistema de control de versiones distribuido ampliamente utilizado [17]. Su uso permitió mantener un registro de los cambios y trabajar de forma incremental.

El repositorio se alojó en GitHub, lo que proporcionó almacenamiento remoto y una copia accesible del proyecto. Además, se utilizó GitHub Pages para publicar la versión web de la aplicación [18]. Este servicio permite desplegar sitios estáticos directamente desde un repositorio, por lo que encaja con la exportación web generada por Godot. La combinación de Godot y GitHub Pages permitió que el resultado final pudiese consultarse desde el navegador sin infraestructura de servidor adicional.

Metodología y planificación

Capítulo 5

Desarrollo

En este capítulo se explicará todo el proceso de desarrollo.

5.1 Mockup: versión 0.1

El paso previo antes de comenzar con un desarrollo profundo consistió en comprobar que las herramientas que se querían usar servían para el propósito propuesto. Fue así que la primera versión “usable” del proyecto consistió en un botón con el que se podía interactuar. Al hacerlo, se mostraría un menú que permitiría añadir nuevos botones, conectados por líneas, formando una estructura con forma de árbol. Esto sería exportado a formato web y se desplegaría en GitHub Pages.

Para iniciar este proceso, primero hizo falta hacer un análisis de la forma en la que se estructura Godot. Esto se explica en la Sección 3.1.

5.1.1 Primeras escenas

La primera escena que se creó fue el menú principal. Al ya esperar crear varios algoritmos, era una escena simple de la que había que partir. En ella se instanció un nodo de tipo Button (un nodo Control (interfaz) caracterizado por emitir una señal cuando es pulsado) con el texto que cambiaba a la escena donde se encontraba el árbol. En la Figura 5.1 se ve este menú principal.

Al pulsar el botón se cambiaba de escena a una en la que se encontraba una entidad de una clase personalizada, el “DNode”. Esta clase funcionaba como un botón (aunque por el momento no heredaba del tipo de nodo Button) y tenía tres atributos más: “id”, “parent_id”, y “sons_id”. Los primeros eran enteros, y el segundo un Array de enteros. Al hacerle click se desplegaba un menú con una opción: añadir nodo hijo. Seleccionarla hacía aparecer un nuevo botón debajo del anterior, conectado por una línea. Esto se podía hacer cuantas veces se quisiera, y hacía que fueran apareciendo nuevos nodos hijo, que se colocaban uno al lado

de otro. Estos siguientes botones tenían una segunda opción en el menú: eliminar nodo. Esto eliminaba el nodo y sus hijos.

Por debajo el árbol era un diccionario. Las claves eran enteros, correspondientes a los ids de los DNodes, y los valores eran los propios DNodes. Estos eran conectados con otros nodos y colocados en pantalla en base a sus atributos de “parent_id” y “sons_id”.

Además, en la escena había otros dos botones arriba a la derecha. En este momento no hacían nada; eran el inicio de lo que acabaría siendo la funcionalidad de cargar y guardar el árbol, retomada durante el desarrollo del árbol de decisión. En la Figura 5.2 se puede ver el estado de esta primera escena interactiva.

5.1.2 Exportación web con GitHub Pages

Con esta versión funcional, faltaba probar a exportar a web para comprobar que Godot servía como motor de desarrollo para la aplicación web. En Godot, para exportar el proyecto a HTML5, hay que descargar las plantillas de exportación, las cuales proporcionan el entorno web base y un archivo HTML que contiene en su estructura el elemento “<canvas>”, donde el motor (compilado en WebAssembly) inicializa el renderizado gráfico y captura la interactividad. La prueba se completó sin problemas con un proyecto tan simple. Más adelante se exploran configuraciones más complejas de la exportación web debido a la creación del plugin para el renderizado de fórmulas.

Después se configuró desde la interfaz web de GitHub la funcionalidad de GitHub Pages. Para ello, accedí desde la configuración del repositorio a la ventana “Pages”, desde la cual se puede seleccionar una rama de publicación (para este caso se seleccionó “main”) y una carpeta para ser la raíz de la página web (en este caso, la carpeta “docs”, la predeterminada). Con esta función activada, hubo que exportar el proyecto a la carpeta indicada. Una vez subidos los cambios a la rama “main”, la aplicación estaba desplegada.

5.2 Árbol de decisión: versión 0.2

El proyecto dejó de ser un simple mockup cuando se comenzó a trabajar en el primer algoritmo: el árbol de decisión.

5.2.1 Primera arquitectura del árbol

Para comenzar el desarrollo del árbol de decisión, el primer objetivo fue separar la prueba visual inicial de una estructura que pudiera sostener más adelante un algoritmo real. La primera versión del árbol, descrita en la sección anterior, servía para crear nodos manualmente y para comprobar que Godot permitía representar una estructura jerárquica en una escena

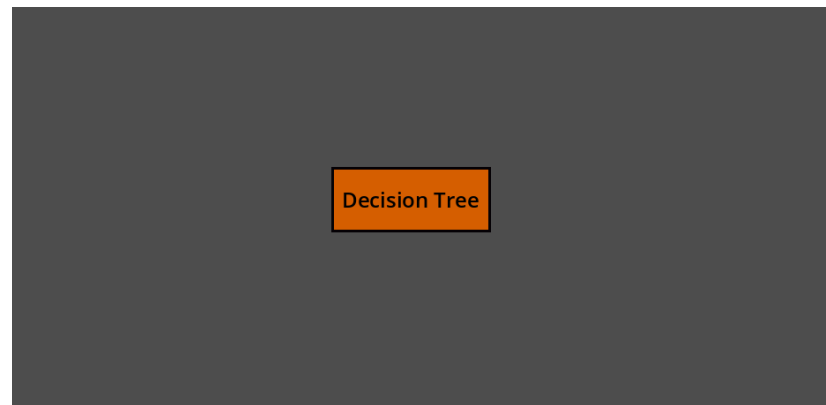


Figura 5.1: Versión preliminar del menú principal.

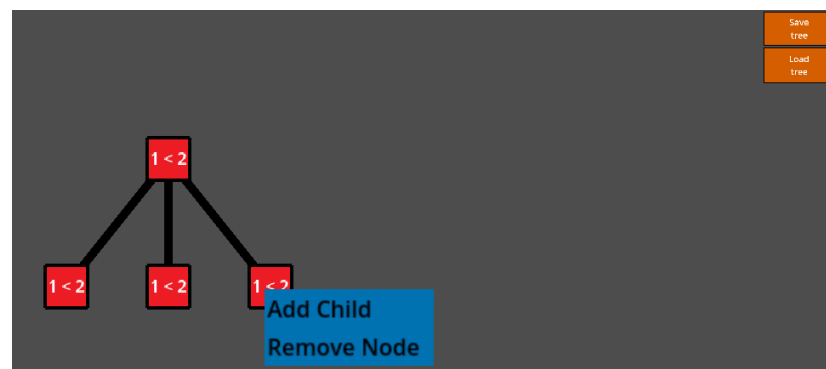


Figura 5.2: Primera escena de prueba para ver el funcionamiento del árbol.

interactiva. A partir de ese punto se empezó a transformar esa prueba en una vista específica de árbol, con sus propias escenas, scripts y responsabilidades.

La escena principal del proyecto quedó configurada en Godot como un menú simple, implementado como un nodo de tipo “Control”. Este menú contenía un único botón que al pulsarse llamaba a “change_scene_to_file” para cargar la escena del árbol. De esta manera, el menú principal podía crecer en el futuro sin mezclar su lógica con la representación del árbol.

La vista del árbol se organizó dentro de un nodo Control de tipo “ScrollContainer”. Este nodo está pensado para tener un solo hijo de tipo Control, y habilita que cuando este es más grande que la zona que tiene asignada, se pueda scrollear para mover el contenido. Esta decisión fue importante desde el principio, porque un árbol puede crecer en anchura y altura con rapidez. Si todos los nodos se colocasen directamente sobre una escena fija, bastarían unos pocos niveles para que parte del árbol quedase fuera de pantalla. Por ello se creó un “DTreeCanvas”, el nodo hijo del ScrollContainer, que instanciaba la escena del árbol. Su función era calcular el tamaño mínimo necesario para contener la estructura completa. El cálculo partía de los límites del árbol, obtenidos mediante “get_tree_bounds”, y ajustaba “custom_minimum_size” para que el ScrollContainer pudiera desplazarse hasta todas las zonas relevantes.

El núcleo visual del árbol se encapsuló en la clase “DTreeLogical”. En su primera versión esta clase heredaba de “Node2D” y mantenía un diccionario “nodes_dict”, donde cada clave era el identificador de un nodo y cada valor era la instancia visual correspondiente. También guardaba “root_id”, que apuntaba al nodo raíz. Al iniciarse, el árbol creaba una raíz con identificador 0, la añadía al contenedor de nodos y ejecutaba un reposicionamiento completo. Esta estructura basada en diccionario permitía acceder a cualquier nodo por su identificador sin recorrer toda la escena de Godot.

El posicionamiento se resolvió mediante un recorrido en orden. Cada nodo guardaba su profundidad y un índice “inorder_index”. Las hojas recibían índices consecutivos, y los nodos internos se colocaban en el punto medio entre el primer y el último hijo. Después, “_apply_positions” multiplicaba esos índices por dos constantes de espaciado, una horizontal y otra vertical. Con ello se conseguía una distribución simple, estable y suficiente para las primeras pruebas: los hijos aparecían por debajo del padre, y el padre quedaba centrado respecto a ellos. Tras cada modificación se llamaba a “relayout_tree”, que recalculaba índices, posiciones y conexiones.

Las conexiones se implementaron como una escena independiente, “Conection”, basada en “Line2D”, un Node2D al que se le pueden establecer un Array de puntos por el que trazar una línea. En un primer momento la línea solo unía las posiciones globales del nodo padre y del nodo hijo. Más adelante se adaptó para trabajar con nodos de tipo “Control”, tomando el centro visual de cada botón y convirtiendo esas posiciones al espacio local de la conexión. El

contenedor de conexiones recorría “nodes_dict”, comprobaba los hijos de cada nodo mediante “sons_id” y creaba una línea por cada relación padre-hijo. Esta separación evitaba que los nodos tuviesen que dibujar sus propias ramas y dejó la responsabilidad de las aristas concentrada en una escena concreta.

El nodo del árbol, “DNode”, también cambió durante esta primera fase. La versión inicial era un “Node2D” con un “Sprite2D”, un “Area2D” y una “CollisionShape2D”, no un nodo Button como tal. El “Area2D” recibía eventos de ratón, cambiaba el color del sprite al pasar el cursor y abría un menú contextual al hacer click. Esta solución funcionaba para una prueba gráfica, aunque obligaba a gestionar manualmente colisiones, sprites y etiquetas. En el refactor posterior el nodo pasó a heredar directamente de “Button”. Con esto se aprovechó el sistema de interfaz de Godot, incluyendo eventos “gui_input”, texto incorporado, temas y tamaños de control.

El “DNode” conservó las variables necesarias para representar la estructura: “id”, “parent_id”, “sons_id”, “depth” e “inorder_index”. A ellas se añadieron variables relacionadas con el aprendizaje automático, como “attribute”, “label”, “is_leaf” y “branch_value”. Aunque al principio muchas de ellas todavía no se usaban para ejecutar un algoritmo completo, dejaban preparado el nodo para distinguir entre nodos internos, que representan atributos de división, y nodos hoja, que representan etiquetas de clasificación. También se mantuvo “can_remove”, usado para impedir que la raíz pudiese eliminarse desde el modo manual.

La interacción manual quedó coordinada por “ControllerDTree”. El controlador recibía referencias al árbol, al canvas y a la vista, calculaba el siguiente identificador disponible y conectaba la señal “change_child_requested” de cada “DNode”. Cuando un nodo solicitaba añadir un hijo, el controlador instanciaba un nuevo “DNode”, le asignaba identificador, padre y profundidad, lo añadía a “nodes_dict”, lo insertaba en el contenedor visual y registraba su identificador en “sons_id” del padre. Cuando se solicitaba eliminar un nodo, el controlador borraba recursivamente todo su subárbol. Para ello duplicaba la lista de hijos antes de iterar, evitando modificar el array original durante el recorrido, eliminaba el identificador del hijo en el padre, retiraba el nodo del diccionario y liberaba la instancia visual con “queue_free”.

En paralelo se crearon dos escenas auxiliares para cargar y guardar árboles. En esta fase eran una funcionalidad preliminar, basada en serializar los atributos estructurales de cada nodo a un archivo JSON en una ruta predeterminada. El guardado recorría “nodes_dict” y extraía identificador, padre, hijos, profundidad e índice de orden. La carga hacía el proceso inverso: leía el JSON, reconstruía instancias de “DNode”, rellenaba sus atributos y las añadía a un nuevo árbol. Aunque esta persistencia se revisaría bastante más tarde, ya muestra que desde el inicio se pensaba en el árbol como una estructura de datos reconstruible, no solo como elementos colocados manualmente en una escena.

El siguiente cambio relevante fue la preparación del modo automático. El controlador

dejó de limitarse a gestionar solo la edición manual del árbol y empezó a recibir un modo de funcionamiento, “manual” o “automatic”. En modo manual seguía conectando señales de nodos y permitiendo crear o borrar ramas. En modo automático se mostraba un menú con botones para iniciar entrenamiento y avanzar al siguiente paso del entrenamiento, basado en el algoritmo ID3. Este menú se añadía sobre la vista principal y conectaba sus botones con métodos del controlador. La interfaz todavía era sencilla, con los Buttons “Start Training” y “Next Step”, y ya introducía el flujo que se mantendría durante el desarrollo: el usuario avanza el algoritmo paso a paso desde la interfaz.

Para ese modo automático se creó la clase “AlgorithmDTree”. Su primera responsabilidad fue implementar una versión básica de ID3 sobre datos categóricos. El algoritmo guardaba los datos de entrenamiento, la lista de atributos disponibles, una referencia al árbol visual y una pila de llamadas, “call_stack”, que permitía convertir una construcción recursiva en una ejecución incremental. Cada contexto de la pila contenía el subconjunto de datos correspondiente, los atributos restantes, el identificador del padre, el valor de la rama por la que se llegaba a ese nodo, la profundidad y un identificador visual todavía pendiente.

El método personalizado “start_training” inicializaba el estado y añadía a “call_stack” el contexto de la raíz. A partir de ahí, cada pulsación en “Next Step” ejecutaba “next_step”, que extraía un contexto de la pila y llamaba a “_build_node_step”. Esta función concentraba la lógica de decisión. Primero obtenía las etiquetas de los datos. Si todas eran iguales, pedía crear un nodo hoja con esa clase. Si no quedaban atributos, calculaba la clase mayoritaria y creaba una hoja de mayoría. En el resto de casos calculaba la ganancia de información para cada atributo, seleccionaba el atributo con mayor ganancia, preparaba un contexto por cada valor posible del atributo elegido y solicitaba crear un nodo interno.

La entropía se calculó directamente en GDScript. La función personalizada “_entropy” contaba ocurrencias de cada etiqueta, transformaba cada conteo en una probabilidad y acumulaba la expresión $-p \log_2(p)$. Sobre esa función se construyó “_information_gain”, que calculaba la entropía total del conjunto, dividía los datos por el valor del atributo y restaba la entropía ponderada de los subconjuntos. Esta implementación fue deliberadamente explícita. En lugar de delegar en una biblioteca externa, el código mostraba de forma transparente los mismos cálculos que después debían explicarse en la interfaz.

La coordinación entre el algoritmo y la escena visual se hizo mediante señales. “AlgorithmDTree” emitía “add_node_requested” indicando padre, valor de rama, atributo, etiqueta y si el nodo era hoja. El controlador recibía esa señal y decidía si debía crear la raíz o un hijo. Para los nodos hijos, actualizaba “branch_value” con el valor del atributo que llevaba hasta ese nodo. Esta variable se usaría poco después para mostrar etiquetas en las conexiones. Como los contextos de los hijos se añadían a la pila antes de saber el identificador real del nodo padre, se introdujo el valor temporal “-2” para marcar padres pendientes. Cuando el contro-

lador terminaba de crear un nodo, llamaba a “update_pending_parent_ids”, que sustituía esos padres pendientes por el identificador recién creado.

Este primer modo automático aún tenía datos de entrenamiento definidos en el propio controlador. El conjunto de prueba contenía frutas descritas mediante atributos categóricos como “color”, “shape” y “size”, y una columna “class”. Aunque era un dataset muy pequeño, cumplía dos funciones. Permitía comprobar que la lógica ID3 generaba nodos y ramas, y permitía observar si el árbol visual se actualizaba correctamente en cada paso.

A partir de aquí el problema principal dejó de ser crear nodos a mano y pasó a ser sincronizar tres niveles: el estado del algoritmo, la estructura lógica del árbol y la escena que veía el usuario.

5.2.2 Reorganización de la vista y primeros elementos explicativos

Una vez creada la primera versión automática, se reorganizó la escena del árbol para

Conclusiones

DERRADEIRO capítulo da memoria, onde se presentará a situación final do traballo, as leccións aprendidas, a relación coas competencias da titulación en xeral e a mención en particular, posibles liñas futuras,...

Capítulo 7

Trabajo futuro

Apéndices

Material adicional

EXEMPLO de capítulo con formato de apéndice, onde se pode incluír material adicional que non teña cabida no corpo principal do documento, suxeito á limitación de 80 páxinas establecida no regulamento de TFGs.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque.

Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Bibliografía

- [1] G. Sanderson, “Neural networks,” 3Blue1Brown, consultado el 8 de junio de 2026. [En línea]. Disponible en: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
- [2] —, “But what is a neural network?” 3Blue1Brown, consultado el 8 de junio de 2026. [En línea]. Disponible en: <https://www.3blue1brown.com/?topic=neural-networks>
- [3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [4] D. Smilkov and S. Carter, “A neural network playground,” TensorFlow, consultado el 8 de junio de 2026. [En línea]. Disponible en: <https://playground.tensorflow.org/>
- [5] M. Kahng, N. Thorat, P. Chau, F. Viégas, and M. Wattenberg, “Gan lab: Play with generative adversarial networks in your browser!” Georgia Tech and Google Brain/PAIR, consultado el 8 de junio de 2026. [En línea]. Disponible en: <https://poloclub.github.io/ganlab/>
- [6] Interactive ML, “Interactive decision trees learning tool,” consultado el 8 de junio de 2026. [En línea]. Disponible en: <https://www.interactive-ml.com/decision-tree.html>
- [7] Godot Engine contributors, “Godot engine,” consultado el 8 de junio de 2026. [En línea]. Disponible en: <https://godotengine.org/>
- [8] —, “Godot documentation,” consultado el 8 de junio de 2026. [En línea]. Disponible en: <https://docs.godotengine.org/>
- [9] —, “Exporting for the web,” consultado el 8 de junio de 2026. [En línea]. Disponible en: https://docs.godotengine.org/en/stable/tutorials/export/exporting_for_web.html
- [10] Python Software Foundation, “Python,” consultado el 8 de junio de 2026. [En línea]. Disponible en: <https://www.python.org/>

- [11] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. VanderPlas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [12] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshain, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [13] The Rust Project Developers, “Rust programming language,” consultado el 8 de junio de 2026. [En línea]. Disponible en: <https://www.rust-lang.org/>
- [14] ratex contributors, “ratex,” crates.io, consultado el 8 de junio de 2026. [En línea]. Disponible en: <https://crates.io/crates/ratex>
- [15] World Wide Web Consortium, “Webassembly core specification,” consultado el 8 de junio de 2026. [En línea]. Disponible en: <https://www.w3.org/TR/wasm-core-2/>
- [16] Mozilla Developer Network, “Javascript,” consultado el 8 de junio de 2026. [En línea]. Disponible en: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [17] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014, consultado el 8 de junio de 2026. [En línea]. Disponible en: <https://git-scm.com/book/en/v2>
- [18] GitHub, Inc., “Github pages documentation,” consultado el 8 de junio de 2026. [En línea]. Disponible en: <https://docs.github.com/en/pages>